Los Alamos
NATIONAL LABORATORY

Distributed, GPU-Aware Programming with FleCSI

FleCSI tutorial

LA-UR-25-32044
2025

NNSA National Nuclear Security Administration
Managed by Triad National Security, LLC,for the U.S. Department of Energy's NNSA.

Module 1

Introduction and Terminology

The Growing Complexity of HPC Systems

- Latest top HPC systems exhibit millions of CPU cores, GPUs from different vendors, and specialized accelerators.
- Increasing system size and heterogeneity makes achieving peak performance extremely challenging.

Rank	System	Cores
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128

Challenges Across System Layers

- Mastering hardware, memory hierarchy, and distributed parallelism demands broad technical expertise.
- Domain scientists risk being diverted from scientific goals by low-level system complexities.
- Continuous collaboration between domain scientists and architectural experts is extremely expensive.

Division of Expertise

Each group to focus on their expertise, achieving both scientific innovation and high efficiency.

- **Domain Scientists:** Develop physical models and simulation codes.
- **Computational Scientists:** Bridge science and computing; design scalable, efficient algorithms.
- **Computer Scientists:** Optimize distributed and parallel execution; interface software with hardware.

Finding a Solution

- A unified approach to target diverse supercomputer architectures with a single, portable codebase.
- A distributed programming model that makes scaling across thousands of nodes straightforward.
- A higher-level abstraction to compose applications efficiently and promote code reuse.
- Clear separation of concerns: dedicated layers to manage parallelism, data movement, and scientific logic independently.

FleCSI is designed to meet exactly these needs.

What is FleCSI?

- enables **single-source performance-portable parallel and distributed codes**.
- **supports multiple backends**: Legion, MPI, and HPX.
- **supports on-node parallelism** for CPU, GPU, and OMP; via Kokkos.
- provides abstractions layers to **allow domain scientists to focus only on the implementation and their data usage**.

Related Work and Positioning

- **Legion**: data-centric, task-based runtime using logical regions and control replication for scalable parallel execution.
  - **FleCSI** leverages Legion as an optional backend while maintaining a unified single-source API across MPI and HPX.
- **StarPU**: heterogeneous task scheduler using explicit data handles and performance models.
  - **FleCSI** compiles task and data registration statically, minimizing runtime overhead and eliminating manual scheduling.
- **Uintah**: component-based multi-physics framework relying on task graphs and AMR.
  - **FleCSI** provides a general task/data abstraction applicable beyond specific physics domains and supports interchangeable backends.

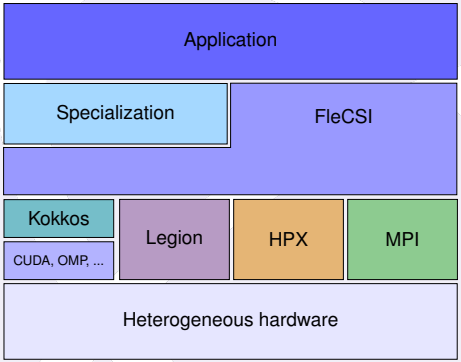
Tutorial Objectives

- Introduction
  - Introduction and Terminology
  - Flog
- Control Model & Runtime Model
  - Actions, Cycles
  - Runtime
- Data Model
  - Concepts (Index Spaces, Fields)
  - Fields (Declare fields, Layouts)
  - Topologies (Register fields)
- Execution Model
  - Tasks (Privileges, Usage)
  - Futures, Reductions
- On-Node Parallelism
  - Executors, Kernels
  - Portable Tasks
- *Composable Interfaces*

FleCSI Terminology

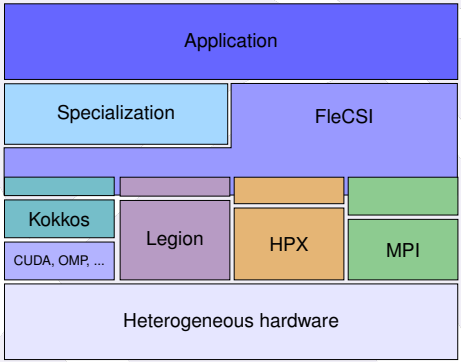
Structure of FleCSI Applications

- **Applications** only care about the physics and the data accesses.
- **Specialization** takes advantage of FleCSI to create domain-specific interfaces.
- **FleCSI** gives interface to lower level with parallel and distributed implementation.



Backends

- **MPI Backend:**
  - Distributed-memory execution across MPI ranks.
  - Implicit synchronization and ghost-data exchange.
- **HPX Backend:**
  - Asynchronous task-based execution with futures.
  - Fine-grained scheduling integrated with on-node parallelism.
- **Legion Backend:**
  - Region-based task model with data locality awareness.
  - Automatic dependency management and profiling support.



Topologies

Many topologies for different use cases:

- **Global**
  - Basic topology to store elements, not partitioned.
- **N-Array**
  - Set of multidimensional arrays.
  - Support for representing a structured mesh.
- **Unstructured**
  - Set of arbitrary multidimentional array and their adjacency maps.
  - Unstructured mesh representation.

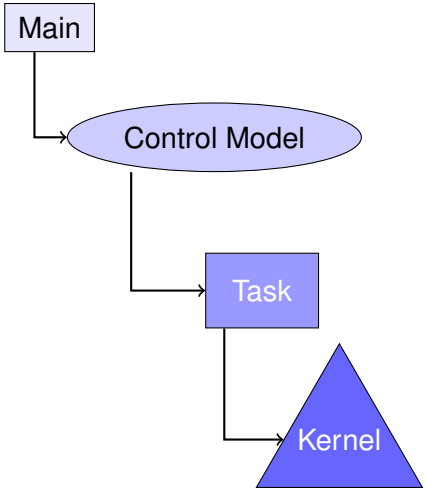
Specializations

- Interface that allows domain experts to implement operations on top of the topology.
- The relevant operations will depend strongly on the needs of the domain experts, but will often include queries about the physical layout of points within the simulation space.

Example

A specialization of the **unstructured topology** for a two-dimensional mesh might define it in terms of vertices, edges, and elements.

Execution Levels



- **Main**: Top level of the application, running simultaneously on each **Process**.
- **Control Model**: FleCSI's generated ordering representing **Actions**. These actions can be executed simultaneously.
- **Task**: An action can launch many tasks. **Colors** can identify which partition is being used in each **Point Task**.
- **Kernel**: The on-node parallelism that can be applied by a point task.

Flog & Runtime

FleCSI Flog: Overview

- Flog is FleCSI's built-in, configurable, parallel logging utility (namespace `flecsi::flog`).
- If disabled at configure time, Flog symbols remain but produce no output.
- Typical use: aggregate task/thread output without interleaving, then serialize/flush periodically.

FleCSI Flog: Severity Levels

```
static auto mytag = flecsi::flog::tag("mytag");

void some_task() noexcept {
  {
    flog::guard guard(mytag);
    flog(trace) << "Trace in mytag" << std::endl;
  }
  flog(info) << "Info" << std::endl;
  flog(warn) << "Warn" << std::endl;
  if (some_condition_failed)
    flog_fatal("Critical error in some_task");
}
```

- **Fatal is special**: `flog_fatal(...)` always emits and aborts, and is not suppressed by tags or strip levels; it can also print a backtrace when enabled.

Running FleCSI

```
#include <flecsi/execution.hh>
#include <flecsi/runtime.hh>

int simulation(flecsi::scheduler &) {
  std::cout << "Hello World" << std::endl;
  return 0;
}

int main() {
  const flecsi::run::dependencies_guard dg;
  flecsi::flog::add_output_stream("clog", std::clog, true);
  return flecsi::runtime().control<flecsi::run::call>(simulation);
}
```

`dependencies_guard` starts the runtime for MPI and Kokkos.
`add_output_stream` describes the output location for flog.
The FleCSI runtime is started on a provided `call` control model.

Building FleCSI and Hands-on #1

Building FleCSI

FleCSI can be built using different methods depending on user needs:

Build Options

- **CMake**: Direct build using installed dependencies.
- **Spack**: Preferred method using the multi-platform package manager for easier dependency handling.

Spack

Hands-on #1: Building FleCSI

In this first hands-on we will

- Access your AWS session that features a pre-configured environment.
- Explore available Spack options for FleCSI.
- Build FleCSI using a prepared Spack installation script.
  - This script is available for your usage after the tutorial.
  - Explore the base code and add the FleCSI runtime, run the program!

We will then begin the next module: Control Model.

Hands-on #1: Building FleCSI

In this first hands-on we will

- Access your AWS session that features a pre-configured environment.
- Explore available Spack options for FleCSI.
- Build FleCSI using a prepared Spack installation script.
  - This script is available for your usage after the tutorial.
  - Explore the base code and add the FleCSI runtime, run the program!

We will then begin the next module: Control Model.

Hands-on #1: Code

As a base for these hands-on exercises, we will use a simple heat–equation solver.

- Each hands-on will extend the initial code as new concepts are introduced.
- The application will compile at every module hands-on.
- The application will not fully execute in Module 4 (by design).
- A complete reference solution is available for each step in the repository if you need to catch up.

Heat Equation solver

Problem Setup

- 2-D heat diffusion on a uniform Cartesian grid.
- Unknown: temperature field $u(x, y, t)$.
- Fixed (Dirichlet) temperatures on all boundaries.
- Initial condition: Gaussian hot spot centered in the domain.

Governing Equation (simple form)

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \qquad \alpha = \text{thermal diffusivity (constant and homogeneous).}$$

Heat Equation solver

Numerical Method

- Spatial discretization: 5-point stencil.
- Time integration: **implicit Backward Euler**

$$(I - \alpha \Delta t \nabla_h^2)\, u^{n+1} = u^n.$$

- Linear system solved with **Red–Black Gauss–Seidel**.
- Convergence checked using:
  - change $\|u^{(k+1)} - u^k\|_\infty$., and
  - residual norm $\|Au^{(k)} - b\|_\infty$.

Heat Equation solver

Verification Using Analytical Solution

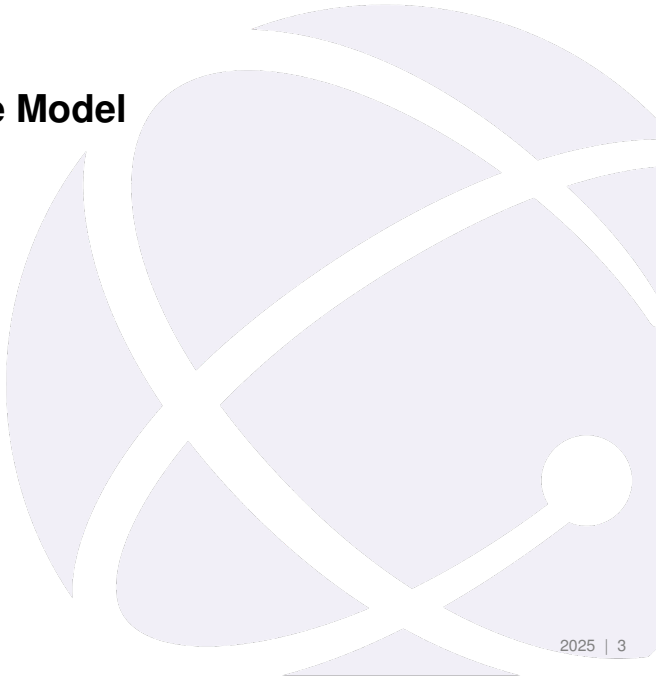
- Gaussian initial condition has an exact solution under diffusion.
- Code computes a relative L^2 error against this exact solution.
- Ensures correctness of discretization and solver.

Outputs

- CSV snapshots of $u(x, y, t)$.
- Can be visualized as 2D plots (python code included)

Module 2

Control Model and Runtime Model



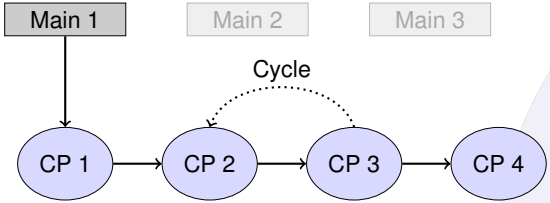
Control Model Overview

- Organizes the application into smaller, modular entities:
  - **Control Points**: Logical stages in the application flow
  - **Actions**: Functions associated with control points, representing specific operations
- **Actions are not tasks**.
  - Actions run sequentially
  - Tasks can be asynchronous
- Benefits of using the Control Model:
  - Simplifies the addition or removal of actions and task groups
  - Enhances code modularity and maintainability

Control Model

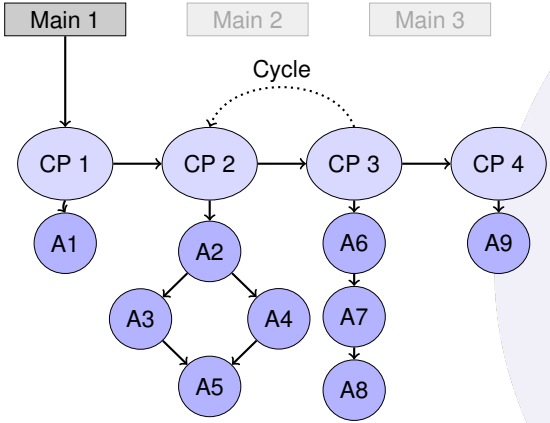
main is **replicated** by each MPI rank, independently of the backend.

Control Model



Control points identify the **high-level stages of the application**. The execution order of the control points is statically defined. Cycles may be defined over any subset of the control points. They form a **control-flow graph** (CFG).

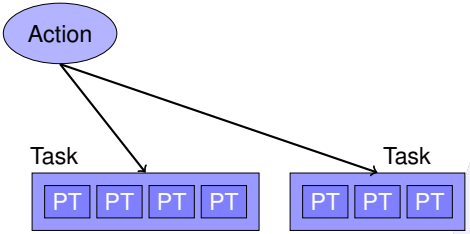
Control Model



Actions are **registered under control points**. Any two actions under a single control point may have a dependency defined between them. The actions under a single control point form a **directed acyclic graph (DAG)**. Actions allow composition of contributed packages. Actions are always executed in a **sequential** program order defined by:

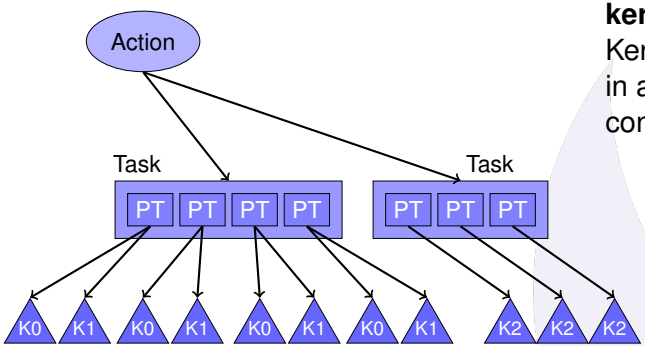
1. control point order
2. topological sorting the actions in the DAG under each control points

Execution Model



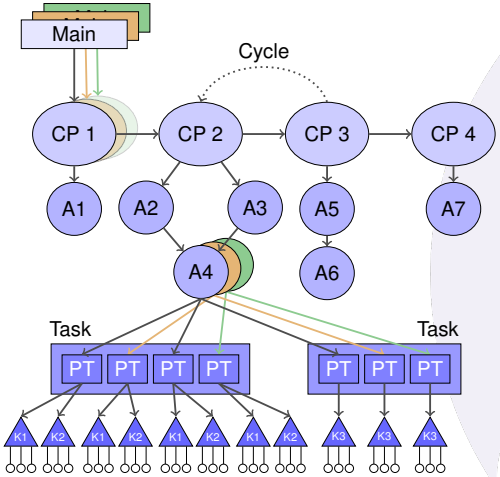
Tasks are executed from inside of an action. Task launch may be **single** or **index**. The elements of a launch are called **Point Tasks**. Tasks typically operate on data that are logically distributed over a partitioned address space. Dependency analysis allows tasks to be executed concurrently by the runtime.

Execution Model



kernels are executed **inside a task**. Kernels execute data-parallel operations in a local address space. Memory consistency of kernel execution is explicit.

Control Model and Execution Model



Runtime Model:
- `main`

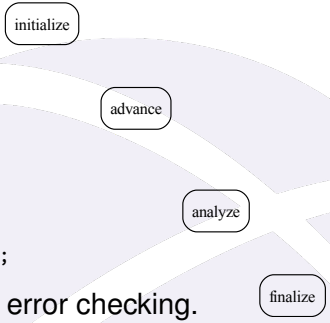
Control Model:
- Control Points
- Actions

Driver:
- Once per process

Execution Model:
- Tasks, Point Tasks, Kernels
- Once per color

Hardware:
- CPU, OMP, GPU: thread

Control Model



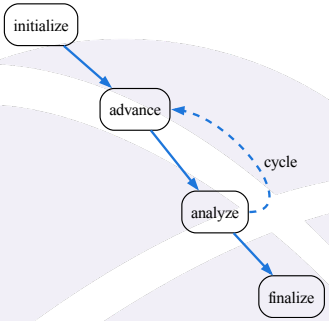
Enumeration defining the control point identifiers.
This will be used to specialize the core control type.

```
enum class cp { initialize, advance, analyse, finalize };
```

Define labels for the control points. Using a function allows error checking.

```
inline const char* operator*(cp control_point) {
  switch(control_point) {
    case cp::initialize: return "initialize";
    case cp::advance: return "advance";
    case cp::analyze: return "analyze";
    case cp::finalize: return "finalize";
  }
  flog_fatal("invalid control point");
}
```

Control Model



Inside the control policy we can define **cycles**:

```
using main_cycle = cycle<cycle_control,
                         point<cp::advance>,
                         point<cp::analyze>>;

static bool cycle_control(control_policy& policy) {
  return policy.step()++ < 5;
}
```

We then select the enumeration of **control point**.
We create the list of control point based on the enumeration values.

```
using control_points_enum = cp;
using control_points = list<point<cp::initialize>,
                            main_cycle,
                            point<cp::finalize>>;
```

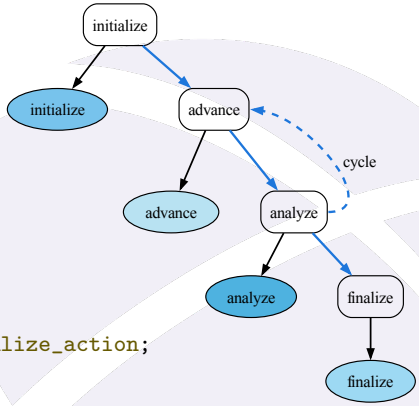
Control Model

We then define the different control point functions.
They then can be registered as an `action`.

```
void initialize(control_policy&) {
  flog(info) << "initialize" << std::endl;
}
control::action<initialize, cp::initialize> initialize_action;
```

Which gives us the output:

```
[info all p0] initialize
[info all p0] advance
[info all p0] analyze
// ...
[info all p0] advance
[info all p0] analyze
[info all p0] finalize
```



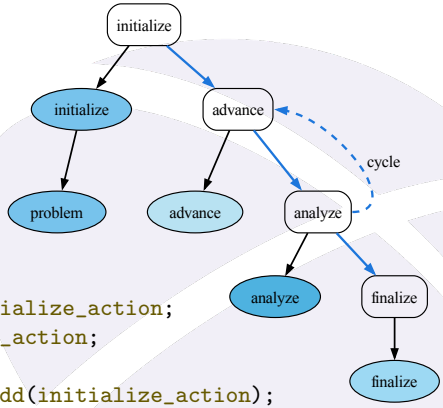
Control Model

We define **dependencies** between actions:

```
void initialize(control_policy&) {}
void problem(control_policy&) {}

control::action<initialize, cp::initialize> initialize_action;
control::action<problem, cp::initialize> problem_action;

inline const auto dep_problem = problem_action.add(initialize_action);
```



Control Model

All these parts are declared inside a `control_policy`:

```
struct control_policy : flecsi::run::control_base {
  // ...
};
```

Once we have created the control policy we can define a fully-qualified control type:

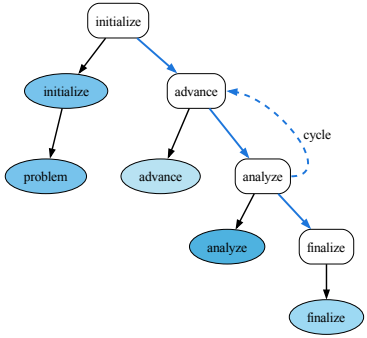
```
using control = flecsi::run::control<control_policy>;
```

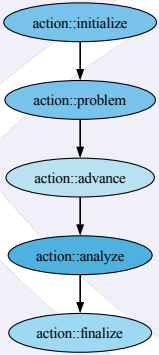
The actions have access to the **control policy**.
It is usually used to store a state containing the needed fields or index spaces for the application.

```
void my_action(control& p) {
  p.my_method();
}
```

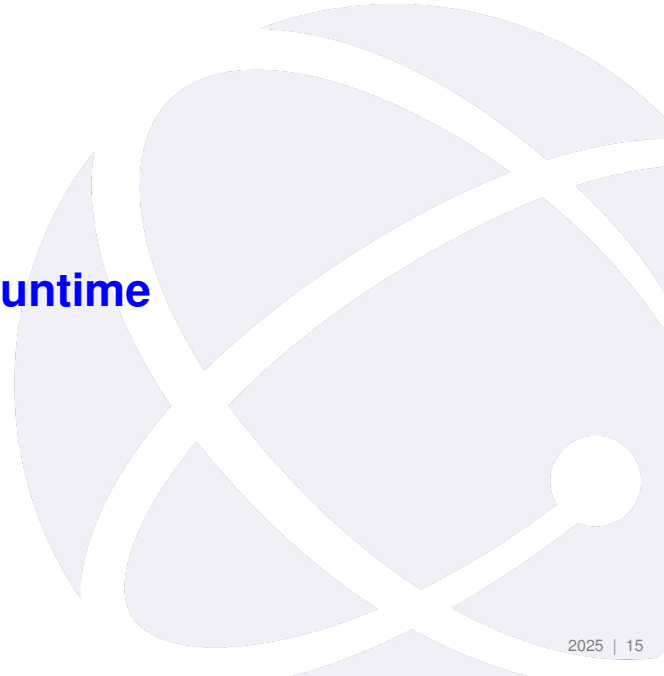
Control Model Visualization

The control model has support for **visualization**. It generates `graphviz` outputs that can be converted to images. It creates a view of the full graph or the selected ordering for the actions:





Runtime



Running FleCSI

Once the control model is implemented, we can run FleCSI on it:

```
int main() {
  const flecsi::run::dependencies_guard dg;
  flecsi::runtime run;
  flecsi::flog::add_output_stream("clog", std::clog, true);
  return run.control<control_model>();
}
```

This will start our control model, running the actions in order.

Program Options

It is possible to create program options with FleCSI, for example, the following adds an option for defining the mesh size:

```
flecsi::program_option<int> mx("Mesh X size",
  "sizex,sx",
  "Specify the size of the mesh in X dimension",
  {{flecsi::option_default, 10}});
```

- The variable `mx` contains the value of the program option.
- If there is a default value it can be accessed with `mx.value()`.
- The user can check for value existance with `mx.has_value()`.

This option can then be used as:

```
./prog --sx=15
```

Hands-on #2

Hands-on #2: Control Model

In this second hands-on we will:

- Add a control model to a base application
- Define actions, cycles, and dependencies
- Update our runtime to use the new control model

Module 3

Data Model

Data Model: Fields and Topologies

- The **data model** defines how simulation data is
  - Declared and registered as **fields**
  - Associated with **topologies** (e.g., meshes, particles)
- **Fields** hold user data:
  - Declared by type and association
  - Registered once and accessed by tasks
- **Topologies** describe the structure of the domain:
  - Define entities (cells, vertices, edges, etc.)
  - Support multiple data distributions and colorings
- Enables **automatic data copy and move**

Concepts

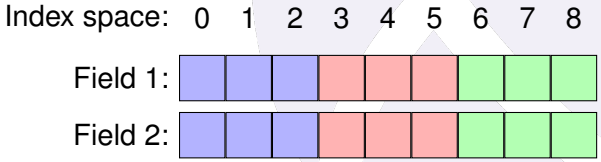
Index Space

- List of entities (cells, nodes, etc.), each with an index point.
- Represents all entities across colors, but
  - Data is distributed and often duplicated (ghost elements).
  - No global ID for access.
- Each point task = 1D array of owned + ghost entities.
- Each point task accesses one color; index point = array index.

Concepts

Fields

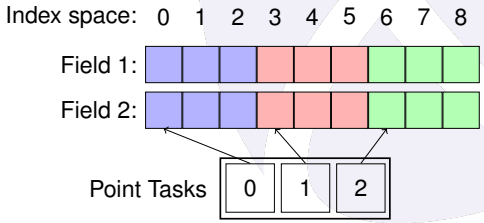
- Defines a variable over an index space (e.g., mass for a cell, momentum for a particle).
- Can represent scalars, vectors, or lists (e.g., per material).
- Manage the memory associated with that variable.



Concepts

Colors

- Subdomains (colors) processed by single C++ function call at a time (point task)
- Not bound to processes; data can move as needed
- With Legion we can have #colors $\neq$ #processes
- Usage examples:
  - Add colors to split expensive regions and avoid idle processors
  - Use fewer colors to work on different physics modules in different processes



Concepts

Ghost elements

- **Ghost elements**: belong to another point task, but copied locally (e.g., for stencil reads)
- **Shared elements**: belong to current point task, are ghosts for others
- **Exclusive elements**: belong to current point task, not shared elsewhere
- Different access permissions for each guide automatic ghost copies
- **Own elements**: Exlusive and Shared elements.
- Different access permissions will guide automatic **ghost copies**

Concepts


color 0 view
extended index space
4 cells
extended index space
9 vertices
color 1 view
extended index space
6 cells
extended index space
13 vertices
color 2 view
extended index space
6 cells
extended index space
13 vertices
color 3 view
extended index space
4 cells
extended index space
9 vertices

Fields

Fields

In the following example we are registering a field on a topology `topo_t`:

```
const flecsi::field<double>::definition<topo_t, topo_t::cells> mass_field;
const flecsi::field<double>::definition<topo_t, topo_t::faces> massflux_field;
```

- We specify the type stored in the field, here `double`
- The `definition` specifies the **topology** and the **index space** on which the field is registered. The first field is stored on the `cells` and the second on the `faces` index spaces that are provided by the topolgogy `topo_t`.

Fields and Data Layout Trade-offs

```
struct cell_data_t { double mass; double energy; };
const flecsi::field<cell_data_t>::definition<topo_t, topo_t::cells> cell_field;
```

Struct of Arrays (SoA)

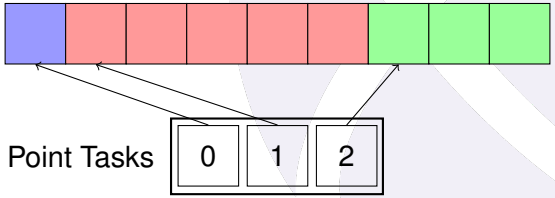
- Enables independent access to each field (e.g., `mass`)
- May incur extra overhead (e.g., dependency analysis, indirections)

Array of Structs (AoS)

- Accessing one field (e.g., `mass`) loads all (e.g., `energy`)
- Trade-offs involve cache usage and vectorization efficiency

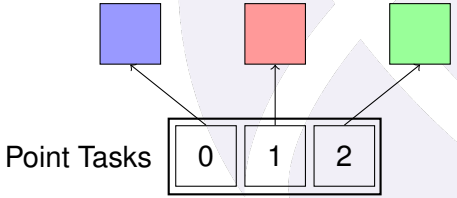
Data Layout: `dense`

- Stores **one value** of type `T` per **index point**
- Most **common layout**; default if none specified
- **Use case**: uniform data per entity (e.g., mass, energy)
- Example: Mass of each cell or momentum of each particle



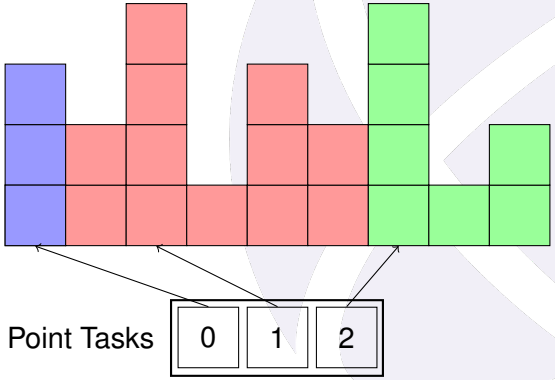
Data Layout: `single`

- Stores **one value** of type `T` per **color**
- Access does **not require an index point**
- **Use case**: flags or settings per subdomain
- Example: Enable a physics operator only for colors that contain relevant cells



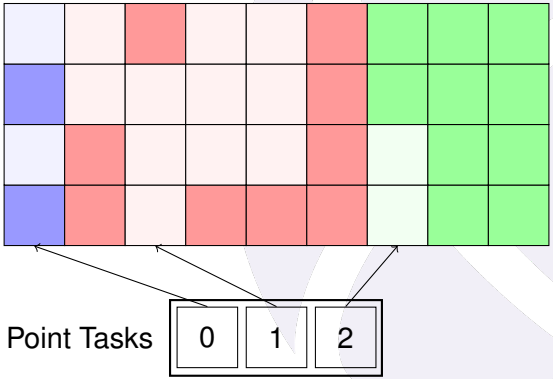
Data Layout: `ragged`

- Stores a **resizable array** of `T` per index point
- Analogous to `std::vector<T>` per index point
- **Use case**: variable-sized data (e.g., material list per cell)
- Materials can **move** or change in count over time



Data Layout: `sparse`

- Stores a **sparse set** of values per index point
- Conceptually like `std::map<std::size_t, T>`
- **More flexible** than `ragged`, allows missing elements
- **Use case**: materials with fixed IDs but not present in all cells



Data Layout: `particle`

- Stores an **unordered set** of `T` per color
- Index points merely serve to distinguish the objects present at any particular time
- **Use case**: tracking dynamic sets of particles
- Particles can move between colors/subdomains

Topologies

Topology

- A **topology** represents the structure of a computational domain (e.g., a mesh, grid, or particle distribution)
- It is composed of one or more **index spaces** that hold elements such as cells, vertices, or particles
- Some index spaces hold **fields** (data), others define **connectivity** or **relationships**
- Applications can use multiple topology instances (same or different types) at once
- Each topology belongs to a **category** (e.g., `unstructured`, `global`) and a **specialization** (e.g., burton, sph)

Basic Topology Types

FleCSI provides several built-in topology types.
The most basic ones do not require specialization:

- **global**
  - Single index space, no color partitioning
  - Ideal for global configuration or small control data
  - No support for `ragged` or `sparse` fields
- **index**
  - Single index space partitioned by color
  - Useful for per-color data such as flags or scalars

Advanced Topology Types

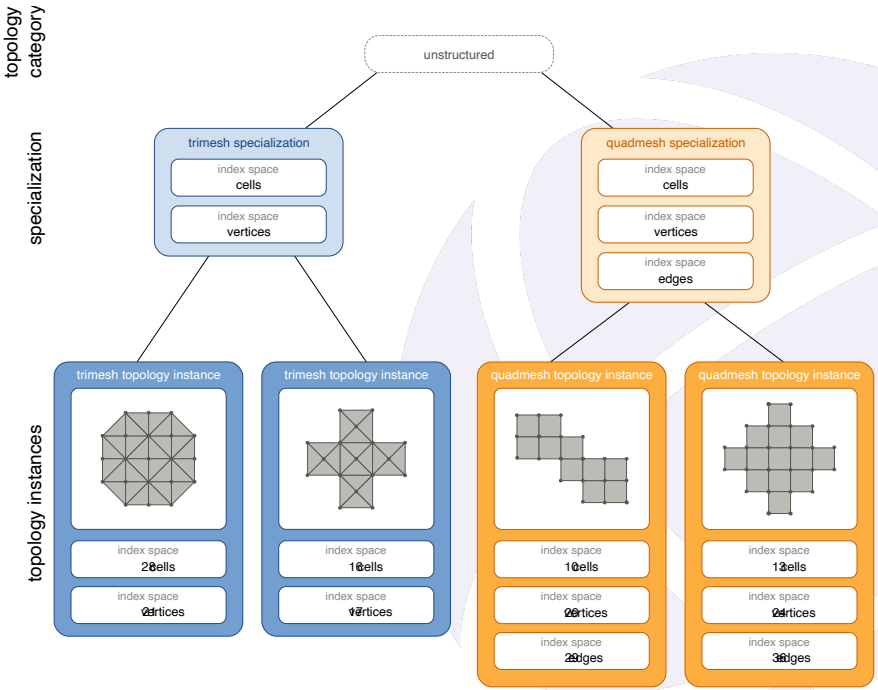
- **user**
  - Single index space sized per color, mutable during execution
  - No ghost support
  - Useful for dynamic per-color data containers

- **narray**
  - Regular, multidimensional Cartesian layout
  - Supports structured meshes with ghost and boundary regions
  - Provides efficient access and connectivity

Advanced Topology Types

- **unstructured**
  - Fully general graph-based topology
  - Arbitrary adjacency relationships
  - Supports mesh refinement

- **ntree**
  - Hierarchical spatial decomposition (binary, quad-, or octree)
  - Ideal for adaptive or particle-based domains
  - Supports neighbor and range queries


topology category
specialization
topology instances
unstructured
trimesh specialization
index space
cells
index space
vertices
quadmesh specialization
index space
cells
index space
vertices
index space
edges
trimesh topology instance
index space
cells
index space
vertices
trimesh topology instance
index space
cells
index space
vertices
quadmesh topology instance
index space
cells
index space
vertices
index space
edges
quadmesh topology instance
index space
cells
index space
vertices
index space
edges

Colorings and Partitioning

- A **coloring** defines how indices are distributed among **colors** (subdomains)
- Used to construct topology instances
- `global` and `index` use just an integer:

```
topo::global::topology pair(s, 2);
topo::index::topology hydro(s, 42);
```

- Complex topologies often uses colorings:

```
canon::topology mesh(s, canon::mpi_coloring(s, "test.txt"));
```

- Topology instances must be created by an action

Field Access

```
auto fr = mass_field(mesh)
```

- Fields are registered on **specializations**; every instance of the associated **topology type** has the field
- Conceptually similar to adding a member to a C++ `struct`
- Returns a **field reference**, which can be passed to **tasks**
- The data contained in fields **can only be accessed in tasks**

- FleCSI automatically allocates memory for fields upon **first access**
- Fields are released when the associated **topology is destroyed**

Topologies and Ghosts

- Some topologies support **ghost (halo) elements** to exchange boundary data between colors
- `narray` and `unstructured` topologies
- Ghosts allow local **computation on shared entities**
- The runtime automatically manages ghost updates before the tasks run
- Access privileges (`ro`, `rw`, `wo`, `na`) control synchronization
- Topology specialization defines which index spaces participate in ghosts

Hands-on #3

Hands-on #3: Fields

In this third hands-on we will

- Use a simple specialization of the N-Array topology
- Add fields on this topology and start our first simple tasks

The exercise files are present in the directory TBD on the machine.

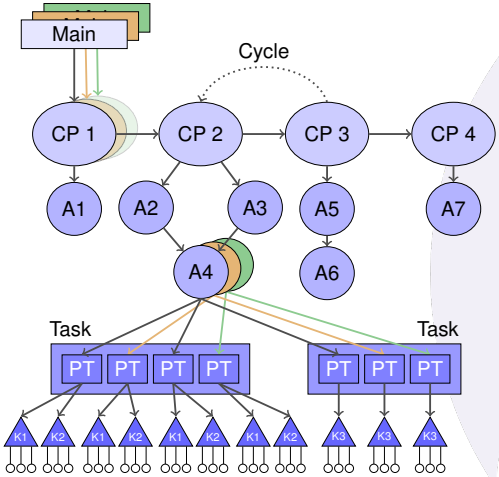
Module 4

Execution Model

Execution Model — Bridging Parallelism Levels

- We have introduced:
  - The **control model**: top-level orchestration
  - The **data model**: fields distributed across topologies
- Task launches bridge the first two levels of the FleCSI's model:
  - **Driver** (main and control model)
  - **Tasks** (distributed parallelism)
- **With tasks**, FleCSI provides interfaces to
  - Declare data usage (privileges)
  - Access data and modify it

Execution Model — Bridging Parallelism Levels



Runtime Model:
- `main`

Control Model:
- Control Points
- Actions

Driver:
- Once per process

Execution Model:
- Tasks, Point Tasks, Kernels
- Once per color

Hardware:
- CPU, OMP, GPU: thread

Tasks

Tasks

- Tasks are **functions** that operate concurrently on a **distributed data structure**
- Task execution involves two sides:
  - **Action side**: provides arguments (e.g., fields, futures)
  - **Task side**: defines parameters (e.g., accessors)
- Argument and parameter types are **not the same**
- **FleCSI automatically converts** action-side arguments into task-compatible parameter types

Task: Action Side

```
int action(control_policy & cp) {
  flecsi::scheduler & s = cp.scheduler();
  s.execute<task>(field1(*spec::ptr), field2(*spec::ptr));
}
```

- Tasks are launched using the `scheduler`
  - available in control model actions
- `execute` takes the task function as a **template parameter**
  - It launches one **point task per color**, each handling a subspace of the data
- Remaining arguments are **field references** and other inputs

Task Signature: Task Signature

```
using namespace flecsi;
template<typename T>
void task(const std::vector<T> &v,
          int i) noexcept
{ /* ... */ }
// ...
std::vector<int> v(...);
s.execute<task<int>>(v, 10);
```

Templates & arguments

- Tasks can be templated
- `std::vector`, values are duplicated on each colors and need to be the same
- Since they are not invoked directly, tasks cannot throw exceptions and must be declared `noexcept`

Task Signature

```
using namespace flecsi;
void task(exec::cpu s,
          field<data>::accessor<rw, na, na> acc_1,
          field<double>::mutator<rw, na, na> mut_2) noexcept
{ /* ... */ }
/* ... */
s.execute<task<int>>(exec::on, /* ... */);
```

Execution Space

- Specify on which execution space the task with execute
- Several options are possible: `cpu`, `gpu`, `omp`, or `accelerator`
- Later we will define a task for multiple execution spaces
- Use `exec::on` on the action side

Task Signature

```
using namespace flecsi;
void task(exec::cpu s,
          field<data>::accessor<rw, na, na> acc_1,
          field<double>::mutator<rw, na, na> mut_2) noexcept
{ /* ... */ }
```

Accessors

- A task accepts **field references** as arguments via **accessors**
- When the task is launched:
  - **Memory** for the fields is allocated if necessary
  - The memory is provided to the task through the accessors

Task Signature

```
using namespace flecsi;
void task(exec::cpu s,
          field<data>::accessor<rw, na, na> acc_1,
          field<double>::mutator<rw, na, na> mut_2) noexcept
{ /* ... */ }
```

Mutators

- **Accessors** can read/write values but **cannot add or remove** them
- Layouts that support dynamic modification use **mutators**
- Mutators exist for Ragged, Sparse, and Particle layouts

Task Signature

```
using namespace flecsi;
void task(exec::cpu s,
          field<data>::accessor<rw, na, na> acc_1,
          field<double>::mutator<rw, na, na> mut_2) noexcept
{ /* ... */ }
```

Privileges

- `na`: no access, `ro`: read-only, `wo`: write-only, and `rw`: read-write
- Accessors and mutators declare the task's **privileges** on each field
- Privileges are used by the runtime to **enforce task order**
- Accessors encode the task's **access privileges**
- First access must use a **write-only mutator** to initialize the field

Privileges

- Topologies with ghosts use **multiple privileges**
  - Privileges control **ghost copies**
- Pointers/references become `const`-qualified if **no write privileges**
- **Undefined behavior** if code writes to regions lacking write permission

Ghost copy

A ghost copy occurs between any two tasks that operate on the same field where

- the first has write access to the shared elements,
- the second has read access to the ghost elements, and
- neither the first nor any intervening task has write access to the ghost elements.

Futures

```
int task(){ return 10;}
int ftask(flecsi::future<int> f){ int v = f.get() }
// ...
flecsi::future<int> f = s.execute<task>();
s.execute<ftask>(f);
```

- A **future** is the return value of a task
- The value of the future is available when the task has actually been executed

Warning

- Calling `f.get()` outside a task forces **serialization** of execution
  - Can significantly impact performance on the Legion backend
- Better practice: **pass futures to other tasks** instead

Reduction

```
auto result = s.reduce<exec::fold::sum>(...);
```

- The `reduce` call performs a reduction across multiple point tasks result.
- It returns a **future** representing the outcome of the reduction operation.
- Users specify the reduction operation by selecting a fold from the `exec::fold` namespace:
  - `sum`, `product`, `min`, and `max`
- You can define a reduction type with
  - `combine`: a function to merge two values.
  - `identity`: the neutral element for the operation.

Further Task Semantic

MPI tasks and the new semantic

Inside the Task

Accessor

```
using namespace flecsi;
void task(exec::cpu,
          field<int>::accessor<rw> acc) noexcept
{
  const auto n = acc.span().size();
  acc[n-1] = 0;
  for(auto& v: acc.span())
    v += 10;
}
```

Accessors

- Can be accessed as an array with `operator[]` or `operator()`
- Provide a `span()` which is an iterable view

Mutator

```
using namespace flecsi;
void task(exec::cpu,
          field<int, data::ragged>::mutator<rw> mut) noexcept
{
  int i = 0;
  for(auto r : mut) {
    r.resize(i, i);
    ++i;
  }
}
```

Mutators

- Usable with `ragged`, `sparse`, and `particle` layout with different syntax.

Hands-on #4

Hands-on #4: Tasks

In this fourth hands-on we will

- Define our own tasks and execute them
- Test reductions and privileges
- Use accessors and mutators
- Add future and pass it to a task

The exercise files are present in the directory TBD on the machine.

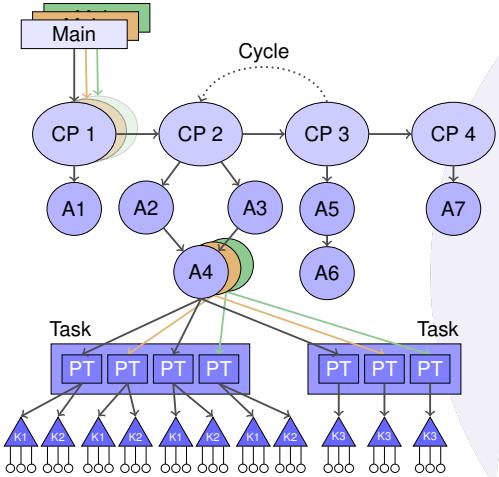
Module 5

On-Node Parallelism

On-Node Parallelism - Getting Performance

- We have introduced:
  - The **control model**: top-level orchestration
  - The **data model**: fields distributed across colors
  - The **execution model**: tasks to distribute and organize work
- **On-node parallelism** support leveraging different accelerators
- It is available through **kernels**
- **Within each task**, FleCSI provides interfaces to use **Kokkos** for
  - Fine-grain, **thread-level parallelism**
  - Efficient use of **hardware backends** (e.g., OMP, CUDA)

Execution Model — Bridging Parallelism Levels



Runtime Model:
- `main`

Control Model:
- Control Points
- Actions

Driver:
- Once per process

Execution Model:
- Tasks, Point Tasks, Kernels
- Once per color

Hardware:
- CPU, OMP, GPU: thread

Executor and Loops

On-Node Parallelism

- FleCSI tasks can launch **kernels** to exploit **fine-grained, on-node parallelism**
- Kernels operate **inside task bodies**
- They are typically mapped to **hardware threads** via **Kokkos**
- The task runs on the host but the data is **placed in the correct memory space**

Executor

The **executor API** provides a consistent interface on execution spaces for

- `forall`: parallel loop
- `reduceall`: parallel reduction

```
void modify(exec::accelerator s, /* ... */ ) noexcept {
  s.executor(). /* ... */
}
```

On-Node Parallelism

FleCSI supports the selection of different execution spaces in the task signature:

- `exec::cpu`: execute on host CPU
- `exec::omp`: execute with OpenMP support
- `exec::gpu`: execute with GPU support
- `exec::accelerator`: depends on the configuration of Kokkos.
  - The architecture will be, if available, `gpu` > `omp` > `cpu`.
  - The `exec::accelerator` is a great choice which allows to test different architectures.
  - If `exec::gpu` or `exec::omp` be used explicitly but not supported by the Kokkos build, the code not compile.

Simple Parallel Loop

```
void modify(exec::accelerator s, field<double>::accessor<rw> p) noexcept {
  s.executor().forall(x, p.span()) {
    x += 1;
  };
}
```

- `forall` executes a **parallel loop** over a range
- Equivalent to `kokkos::parallel_for`
- `x` is the implicit loop variable
- `p.span()` defines the iteration space
- The loop body is a **lambda expression** ending with a semicolon

Reductions Kernels

```
void reduce(exec::accelerator s, field<float>::accessor<ro> p) noexcept {
  auto res = s.executor().reduceall(
    x, up, p.span(), exec::fold::max, float) {
      up(x);
    };
}
```

- `reduceall` performs a **parallel reduction**
- Based on `Kokkos::parallel_reduce`
- `up` is a reducer variable of type `float`, it is a function `up(value)`
- Reduction operation defined by `exec::fold::max`, `sum`, `min`, and `product`

Ranges

FleCSI provides different tools to iterate on fields, they are all **GPU compatible**.

```
void task(exec::cpu, field<float>::accessor<ro> p) noexcept {
  for(auto v: p.span())
    v += 1
}
```

span

- Can be retrieved directly from accessor/mutator
- Non-owning view similar to `std::span` from C++20
- Provide access to 1D array with any container providing size and pointer

`std::ranges::iota_view` can be used to make a range of integers

Advanced Ranges

```
void task(exec::cpu, field<float>::accessor<ro> p) noexcept {
  int is = 3, js = 4;
  flecsi::util::mdspan<float, 2> mds(p.span(), {js, is});
  for(int i = 0; i < is; ++i)
    for(int j = 0; j < js; ++j)
      mds(i, j) = i+j;
}
```

mdspan and mdcolex

- Built on top of the `span` and based on a subset of `std::mdspan` from C++23
- Multi-dimensional indexing
- `mdcolex` is similar to `mdspan` with reversed indices

Advanced Ranges

The range can be used in combination with the `forall` or `reduce` loops:

```
flecsi::util::mdspan<float, 2> mds(p.span(), {js, is});
s.executor().forall(
  mi,
  mdiota_view(
    mds,
    exec::full_range,
    exec::prefix_range{2}
  )) {
    auto [i, j] = mi;
    mds[j][i] = i*j;
  };
```

- `mdiota_view` provided, like `std::ranges::iota_view` for multidimensional arrays
- Range selectors from `mdiota_view`: `full_range`, `prefix_range`, and `sub_range`

Executor Features

```
    s.executor()
      .threads<64, 1>()
      .named("named forall")
      .for_each(p.span(),
        [p] FLECSI_INLINE_TARGET (auto c) {
          p[c] += 2;
        });
  }
```

- `threads<64, 1>()`: specifies number of threads/blocks:
  - TODO: EXPLAIN

Executor Features

```
s.executor()
  .threads<64, 1>()
  .named("named forall")
  .for_each(p.span(),
    [p] FLECSI_INLINE_TARGET (auto c) {
      p[c] += 2;
    });
```

- `named("...")`: labels a kernel (useful for debugging and profiling)
- `for_each`: function-object version of `forall`, `reduce` of `reduceall`
  - Compared to `forall`, `for_each` allows to specify the lambda and its capture/parameter
- Device functions need to be annotated with `FLECSI_INLINE_TARGET`

GPU Pitfalls

```
void task(exec::gpu s, field<float>::accessor<rw> p) noexcept {
  p[0] = 0; // Error: cannot be accessed on the host
  std::vector<int> v{1,2,3,4};
  s.executor().forall(x, p.span()) {
    x += v[0]; // Error: host std::vector cannot be accessed
  };
}
```

Caution!

- The field data is put on the GPU for you and is not accessible on the host
- Data declared outside the `forall` are copied, if possible (trivially copyable)
- You can use the `for_each` construct to control the capture list

Portable Tasks

Portable Tasks

- A **portable task** is composed of several **variants** targeting different **execution spaces**
- Variants are defined as **specializations** of a static member function template
- FleCSI automatically selects a valid variant when the task is launched
- Variants that are not implemented can be explicitly deleted to prevent their use

Task Template Structure

```
struct task_variants {
  template<class S>
  static void task(S, /* ... */ ) noexcept {
    /* ... */
  }
};
```

- The name `task` for the function is predefined
- It defines the generic interface for the task
- For technical reasons, portable tasks **cannot** be implemented as non-member or overloaded functions

Example: Portable Task Variants

```
struct task_variants {
  template<class S>
  static void task(S, /* ... */ ) noexcept = delete;
};

template<>
void task_variants::task(exec::cpu c, /* ... */ ) noexcept { /* ... */ }

template<>
void task_variants::task(exec::omp c, /* ... */ ) noexcept { /* ... */ }
```

- The primary `task` template is deleted to remove the default implementation
- Specialized variants are provided for `exec::cpu` and `exec::omp`
- Tasks cannot be launched on other architectures (e.g., `exec::gpu`)

Hands-on #5

Hands-on #5: on-node parallelism

In this last hands-on we will:

- Use the FleCSI parallel loops (forall and reduce)
- Use advanced ranges
- Use a portable task

The exercise files are present in the directory TBD on the machine.

Conclusion

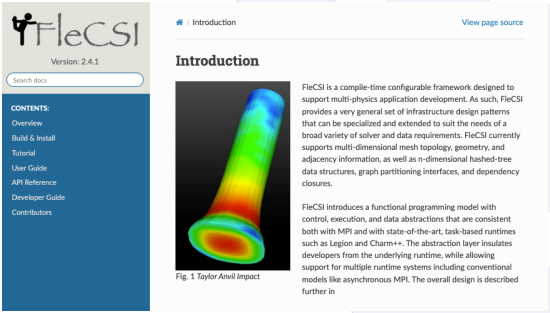
Conclusion slide

Show full model again, explain that we didnt speak about how to write specializations.

Resources

Available Resources:

- `https://github.com/flecsi/flecsi`: FleCSI github repository
  - Create **issue** or use the **GitHub discussion board**
- `https://flecsi.org`:
  - FleCSI Tutorial + User guide
  - User API Reference



The FleCSI Team

Davis Herring

Ben Bergen

Scott Pakin

Maxim Moraru

Richard Berger

Julien Loiseau

Los Alamos NATIONAL LABORATORY

FleCSI contributors

Benjamin Bergen, Nick Moss, Irina Demeshko, Davis Herring, Marc Charest, Julien Loiseau, Navamita Ray, Jonathan Graham, Hartmut Kaiser, Li-Ta Lo, Karen Tsai, Charles Ferenbaugh, Richard Berger, John Wohlbier, Jonas Lippuner, Wei Wu, Andrew Reisner, Christoph Junghans, Scott Pakin, Brendan K. Krueger, Lukas Spies, Sumathi Lakshmiranganatha, Max Ortner, Pascal Grosset, David Gunter, Maxim Moraru, Galen Shipman, Jiajia Waters, Scot Halverson, Onur Çaylak, Peter Brady, Philipp V. F. Edelmann, Mason Delan, Brandon Keim, Christopher Malone, Alex Villa, Daniel Holladay, Dani Barrack, Nikunj Gupta, Ondřej Čertík, Robert Bird, Melissa Rasmussen